Guia rápido · Módulo 02: Fundamentos de JavaScript

Resumo para consulta rápida. A explicação completa está nos slides e nos arquivos desta pasta.

Variáveis e tipos

```
let idade = 20          // pode mudar
const nome = "Ana"      // não pode ser reatribuída
typeof "oi"             // "string"
typeof 42               // "number"
typeof true             // "boolean"
typeof undefined        // "undefined"
typeof null             // "object" (herança antiga do JS)
Array.isArray([1, 2])   // true (typeof [] é "object")
`Olá, ${nome}!`         // template string
```

Entrada com prompt-sync

```
const prompt = require("prompt-sync")()
const n = Number(prompt("Digite um número: ")) // prompt devolve TEXTO
isNaN(n)          // true se não virou número
parseInt("12.9")  // 12
parseFloat("2.5") // 2.5
```

Operadores

`+ - * /` · `%` resto · `**` potência · `++` `--` · `+=` `===` igual (valor **e** tipo) · `!==` diferente · `>` `>=` `<` `<=` `&&` e · `||` ou · `!` não · ternário: `cond ? a : b`

Falsy: `false`, `0`, `""`, `null`, `undefined`, `NaN`. O resto é truthy.

Decisão

```
if (nota >= 7) {
  console.log("Aprovado")
} else if (nota >= 5) {
  console.log("Recuperação")
} else {
  console.log("Reprovado")
}

switch (opcao) {
  case "1": {
    console.log("Somar")
    break          // sem break, cai no próximo case
  }
  default:
    console.log("Opção inválida")
}
```

Repetição

```
for (let i = 0; i < 5; i++) { }
while (condicao) { }           // pode nem rodar
do { } while (condicao)        // roda pelo menos uma vez
for (const item of lista) { }  // cada valor do array
for (const chave in objeto) { }// cada chave do objeto
break     // sai do laço
continue  // pula para a próxima volta
```

Arrays

Método	Faz
`push(x)` / `pop()`	põe / tira no fim
`unshift(x)` / `shift()`	põe / tira no começo
`length`	tamanho
`includes(x)` / `indexOf(x)`	tem? / posição (−1 se não tem)
`slice(i, f)`	cópia de um pedaço
`splice(i, n)`	remove `n` itens a partir de `i`
`join(", ")`	vira texto
`sort((a, b) => a - b)`	ordena números

Strings

`length` · `toUpperCase()` · `toLowerCase()` · `trim()` · `split(" ")` · `includes("x")` · `slice(0, 3)` · `replace("a", "b")` · `texto[0]`

Objetos

```
const aluno = { nome: "Ana", nota: 8 }
aluno.nome            // "Ana"
aluno["nota"]         // 8
aluno.turma = "LD1"   // cria a chave
Object.keys(aluno)    // ["nome", "nota", "turma"]
```

Funções

```
function somar(a, b) {
  return a + b
}
const dobro = (n) => n * 2
function saudar(nome = "visitante") { return `Oi, ${nome}` }
```

Debug no VS Code

Clique na margem para o **breakpoint** (bolinha vermelha) → **F5** → **F10** avança uma linha, **F11** entra na função. Veja os valores no painel **Variables**.

Armadilhas

- `"2" + 3` → `"23"`. Converta a entrada com `Number()`.
- `=` atribui; `===` compara.
- `0.1 + 0.2` → `0.30000000000000004`. Para exibir: `toFixed(2)` (devolve **texto**).
- `[10, 9, 1].sort()` → `[1, 10, 9]`: sem comparador, ordena como texto.
- Sem ponto e vírgula, uma linha que começa com `(` ou `[` gruda na anterior.
- `const` com objeto: a variável não muda, mas as propriedades sim.
- `while` sem atualizar a condição = laço infinito (**Ctrl+C** para parar).